AMALITECH CODESIGNAL PRACTICE PACK

Mock Technical Assessment for Candidate Preparation

Prepared for WRP / ITO candidates | Practice before the real Amalitech assessment

Why this practice pack matters

The expected pattern is: early questions are easier, later questions are more difficult, and matrix/grid robot movement questions may appear. Use this document to practice speed, accuracy, and problem-solving structure.

Practice Instructions

- Try the questions in order, but do not get stuck too long on one problem.
- Start by writing a simple working solution. Optimization can come after the basic solution passes tests.
- Use JavaScript, Python, Java, or any language you are comfortable with on CodeSignal/onlinegdb
- For every question, test your solution with the sample input and also create your own extra test cases.
- If you cannot finish a hard question, write partial logic. On CodeSignal, partial progress can still help your score.

Target mindset for tomorrow

Secure the easier points first. Complete the easier array/string questions quickly, then move to matrix/grid and path problems. The goal is not to solve every hard question perfectly, but to collect as many points as possible with clean, working code.

Question Blueprint

#	Question	Difficulty	Main Skill	Why It Helps
1	First Unique Character	Easy	String + HashMap	Appeared in previous WRP style
2	Most Common Word	Easy	Arrays + Frequency	Similar to easy array/word question
3	Two Sum	Easy-Medium	HashMap	Common CodeSignal pattern
4	Valid Anagram Groups	Medium	String normalization	Builds frequency logic
5	Maximum Coins in Grid	Medium	Matrix + Dynamic Programming	Prepares for robot/grid movement
6	Robot Grid with Traps	Medium-Hard	Matrix + DP	Closest to described real exam pattern
7	Decode Nested String	Medium-Hard	Stack	Used in previous practice
8	Shortest Path in Maze	Hard	BFS + Queue	Hard matrix traversal
9	Longest Consecutive Sequence	Hard	Set / Hashing	Fast array reasoning
10	Maximum Score Path with Diagonal	Hard	DP + Matrix	Advanced grid scoring

Question 1: First Unique Character

Difficulty: Easy	Points: 40	Focus: String, frequency map	Time box: 10-18 min

Given a string s, return the index of the first character that does not repeat. If every character repeats, return -1.

Input

```
s = "leetcode"
```

Expected Output

```
0
```

Example

```
Because "l" appears only once and it is at index 0.
```

Constraints / Clarifications

- The input contains lowercase English letters only.
- Think about counting each character first, then scanning again.

Student tip

Use a dictionary/hash map to count letters. Then loop from left to right and return the first index whose count is 1.

Question 2: Most Common Word

Difficulty: Easy	Points: 40	Focus: Array, words, frequency	Time box: 10-18 min

Given an array of words, return the word that appears most frequently. If there is a tie, return the word that appears first in the array.

Input

```
words = ["java", "python", "java", "javascript", "python", "java"]
```

Expected Output

```
"java"
```

Example

```
java appears 3 times, python appears 2 times, javascript appears 1 time.
```

Constraints / Clarifications

- Words are case-sensitive unless your language is told otherwise.
- Use the first occurrence to break ties.

Student tip

This is a very likely easy question style: count values in an array and keep the maximum.

Question 3: Two Sum

Difficulty: Easy-Medium	Points: 50	Focus: Array, hash map	Time box: 10-18 min

Given an array of integers nums and a target, return the indices of two numbers that add up to the target. Assume exactly one valid answer exists.

Input

```
nums = [2, 7, 11, 15]
target = 9
```

Expected Output

```
[0, 1]
```

Example

```
nums[0] + nums[1] = 2 + 7 = 9.
```

Constraints / Clarifications

- Return indices, not values.
- You cannot use the same element twice.

Student tip

Use a hash map: for each number, check if target - number was seen before.

Question 4: Valid Anagram Groups

Difficulty: Medium	Points: 55	Focus: String, sorting/hash map	Time box: 10-18 min

Given an array of words, group words that are anagrams of each other. Return the groups in any order.

Input

```
words = ["eat", "tea", "tan", "ate", "nat", "bat"]
```

Expected Output

```
[["eat","tea","ate"], ["tan","nat"], ["bat"]]
```

Example

```
Words with the same letters belong together.
```

Constraints / Clarifications

- The order of groups does not matter.
- The order inside each group does not matter.

Student tip

Sort each word to create a key. Example: eat, tea, ate all become aet.

Question 5: Maximum Coins in Grid

Difficulty: Medium	Points: 60	Focus: Matrix, dynamic programming	Time box: 10-18 min

A robot starts at the top-left cell (0,0) of a grid. It can only move right or down. Each cell contains coins. Return the maximum coins the robot can collect when reaching the bottom-right cell.

Input

```
grid = [
  [1, 3, 1],
  [1, 5, 1],
```

```
  [4, 2, 1]
]
```

Expected Output

```
12
```

Example

```
Best path collects 1 + 3 + 5 + 2 + 1 = 12.
```

Constraints / Clarifications

- Robot can only move right or down.
- Include the starting and ending cell values.

Student tip

This prepares candidates for grid/robot movement. Build a DP table where each cell stores the best score up to that cell.

Question 6: Robot Grid with Traps

Difficulty: Medium-Hard	Points: 75	Focus: Matrix, traps, dynamic programming	Time box: 10-18 min

A robot starts at (0,0). It can move right or down. Cells with -1 are traps and cannot be entered. Other cells contain score values. Return the maximum score the robot can collect before reaching the bottom-right cell. If no valid path exists, return -1.

Input

```
grid = [
  [1, 3, 1],
  [1, -1, 1],
  [4, 2, 1]
]
```

Expected Output

```
9
```

Example

```
Best valid path: 1 -> 1 -> 4 -> 2 -> 1 = 9.
```

Constraints / Clarifications

- Trap cells are blocked.
- If start or end is a trap, return -1.
- Only right and down movements are allowed.

Student tip

This is closest to the candidate-described real exam pattern: matrix/grid, robot movement, coordinates, traps, and maximum score.

Question 7: Decode Nested String

Difficulty: Medium-Hard	Points: 70	Focus: Stack, strings	Time box: 10-18 min

Given an encoded string where k[letters] means repeat letters k times, decode the final string. Nested patterns are allowed.

Input

```
s = "3[a2[c]]"
```

Expected Output

```
"accaccacc"
```

Example

```
a2[c] becomes acc, then 3[acc] becomes accaccacc.
```

Constraints / Clarifications

- k can be more than one digit, for example 12[a].
- Nested brackets are valid.

Student tip

Use two stacks: one for repeat counts and one for previous strings.

Question 8: Shortest Path in Maze

Difficulty: Hard	Points: 80	Focus: BFS, queue, matrix	Time box: 10-18 min

You are given a grid where 0 means open space and 1 means wall. A robot starts at (0,0) and wants to reach the bottom-right cell. It can move up, down, left, or right. Return the minimum number of moves required. If impossible, return -1.

Input

```
grid = [
  [0, 0, 0],
  [1, 1, 0],
  [0, 0, 0]
]
```

Expected Output

```
4
```

Example

```
One shortest path is right, right, down, down.
```

Constraints / Clarifications

- Do not walk through walls.
- Do not visit the same cell repeatedly.
- Moves outside the grid are invalid.

Student tip

Use BFS because it finds the shortest path in an unweighted grid.

Question 9: Longest Consecutive Sequence

Difficulty: Hard	Points: 65	Focus: Set, array	Time box: 10-18 min

Given an unsorted array of integers, return the length of the longest consecutive sequence.

Input

```
nums = [100, 4, 200, 1, 3, 2]
```

Expected Output

```
4
```

Example

```
The longest consecutive sequence is 1, 2, 3, 4.
```

Constraints / Clarifications

- The array may contain duplicates.
- Aim for an efficient solution using a set.

Student tip

Only start counting from a number if number - 1 is not in the set. That means it is the beginning of a sequence.

Question 10: Maximum Score Path with Diagonal Move

Difficulty: Hard	Points: 85	Focus: Dynamic programming, matrix	Time box: 10-18 min

A robot starts at the top-left cell and must reach the bottom-right cell. It can move right, down, or diagonal down-right. Each cell has positive or negative score. Return the maximum score possible.

Input

```
grid = [
  [1, 2, 3],
  [4, -5, 6],
  [7, 8, 9]
]
```

Expected Output

```
29
```

Example

```
One strong path is 1 -> 4 -> 7 -> 8 -> 9 = 29.
```

Constraints / Clarifications

- Diagonal move goes from (r,c) to (r+1,c+1).
- Negative cells are allowed, not traps.
- Include start and end cell values.

Student tip

This is an advanced version of robot-grid scoring. Compare three previous options: top, left, and top-left diagonal.

Final Revision Checklist

- I can count character or word frequency using a hash map/dictionary.
- I can solve Two Sum using a hash map.
- I understand matrix coordinates: row starts at 0, column starts at 0.
- I can move through a grid using row and column changes.

- I understand the difference between a trap/wall and a negative score cell.
- I can use dynamic programming for maximum path score.
- I can use BFS for shortest path in a grid.
- I can test my code using simple examples before submitting.
- I will not spend too long on one hard question before securing easier points.